King Abdulaziz University

EduGate: Multilingual School Management System

Prepared By:

Name	ID
Hashim Bajaber	2336321
Abdulaziz Melebari	2338705
Rami Alyazedy	2343138
Abdoul Malick Cisse	2250954
Munthir Alfarsi	2340042

Section: IT2

CPIT-251

Present to: Prof. Awny Sayed

Table of Contents

Contents

Abstract....3

Introduction 3

Problem Statement....3

Proposed Solution....3

Approach (Software Development Process)....4

Requirements + System Modelling & Analysis 5

Diagrams: 6

Software Architecture (Layered Architecture)....13

Layers: 14

User....17

Methods:....17

Methods:....17

Implementation....21

The methods running on example inputs:....22

The outputs generated by the code: 23

Testing....24

Testing result:....27

User Interface Loging:....28

Team Synergy & Work Distribution....31

Challenges & Lessons Learned....32

Conclusion & Future Goals 33

Abstract

EduGate is a multilingual school management system designed to automate daily school operations and communication among administrators, teachers, students, and parents. It supports Arabic, English, and French, making it ideal for diverse educational environments.

Introduction

Traditional school management relies heavily on paper-based processes, causing delays, human errors, and weak communication. EduGate addresses these issues through a digital, role-based system that integrates attendance, grading, announcements, assignments, and fee management into one unified platform

Problem Statement

Schools in Mali and similar regions struggle with outdated manual systems for recording attendance, managing grades, and communicating with parents. These issues result in inefficiency, poor data tracking, and limited visibility of student progress

Proposed Solution

EduGate provides a centralized multilingual platform with modules for attendance, grades, assignments, communication, and finance. It automates school tasks, enhances accuracy, supports real-time notifications, and generates reports for better decision-making

Approach (Software Development Process)

1. Agile Methodology Implementation We utilized the **Agile methodology** following an iterative and incremental model. Development was segmented into sprints, prioritizing core modules like **Attendance** and **User Management** before extending to complex subsystems like **Finance**.

2. Justification Agile was critical for managing the complexity of EduGate's **Layered Architecture** and trilingual support (Arabic, English, French). It allowed for rapid prototyping, enabling us to validate **Domain Layer** business rules—such as grading logic and attendance policies—directly against feedback from real schools.

3. Technical Benefits

- **Risk Mitigation:** Iterative delivery supported a **gradual rollout**, allowing schools to operationalize core features while advanced modules were still in development.

- **Quality Assurance:** Continuous integration during sprints ensured early error detection within the **Application Layer**, preventing logic errors from propagating to the **Data Layer**.

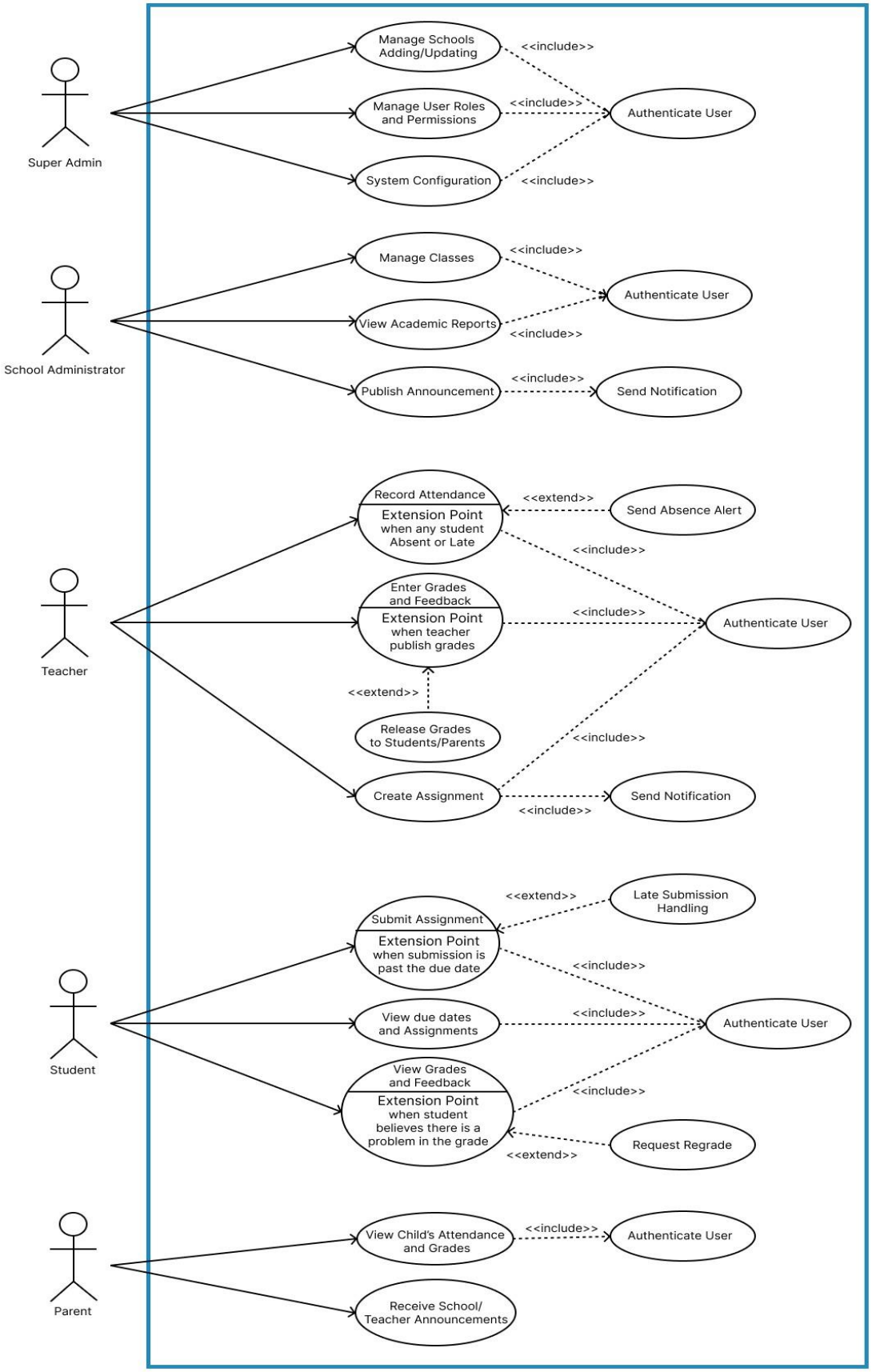
Requirements + System Modelling & Analysis

Functional Requirements

- **Attendance Management:** Record attendance daily or per class; generate alerts for absences.
- **Grades and Evaluation:** Input, calculate, and publish grades; generate performance reports.
- **Assignments:** Teachers post assignments; students upload submissions; automatic late detection.
- **Communication:** Announcements, messaging, and push notifications for students and parents.
- **Financial Management:** Manage school fees, scholarships, and salaries.
- **Reports:** Generate academic, attendance, and financial reports periodically.
- **User Roles:** Super Admin, School Administrator, Teacher, Student, and Parent

Diagrams:

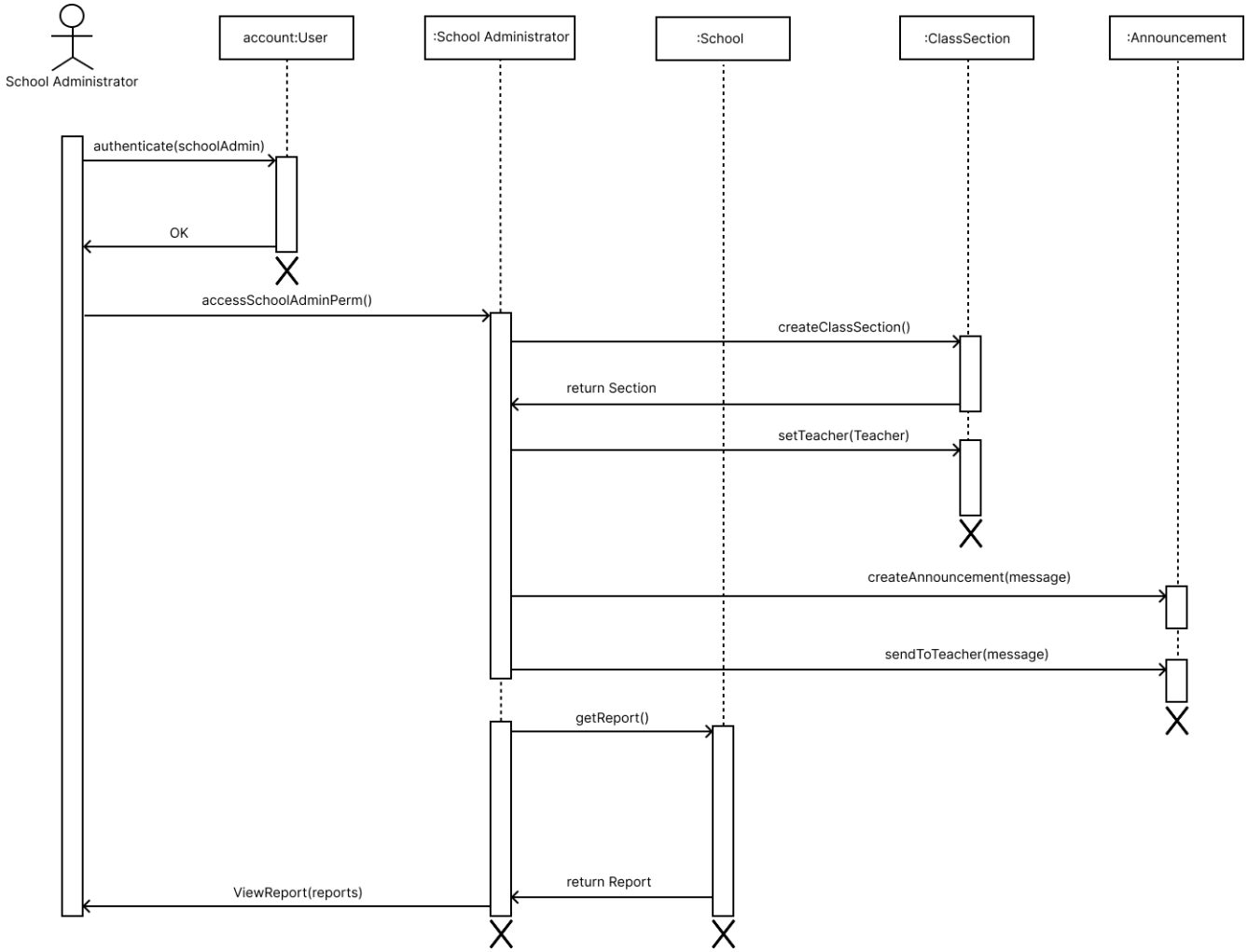
Use Case Diagram



Actor	Main Use Cases	Short Description
Super Admin	Manage Schools, Roles, and System Configs	Adds, updates, or removes schools; defines user roles; updates system settings.
School Administrator	Manage Classes, Teachers, and Announcements	Creates class sections, assigns teachers, publishes school announcements, and views reports.
Teacher	Manage Attendance, Assignments, and Grades	Records attendance, creates assignments, grades submissions, and releases grades to students.
Student	View Assignments and Grades; Submit Work	Views schedule and assignments, submits work, checks grades, and requests regrades when needed.
Parent	Track Child Progress and Receive Alerts	Views attendance and grades of their children and acknowledges school announcements.

Sequence Diagrams

School Administrator

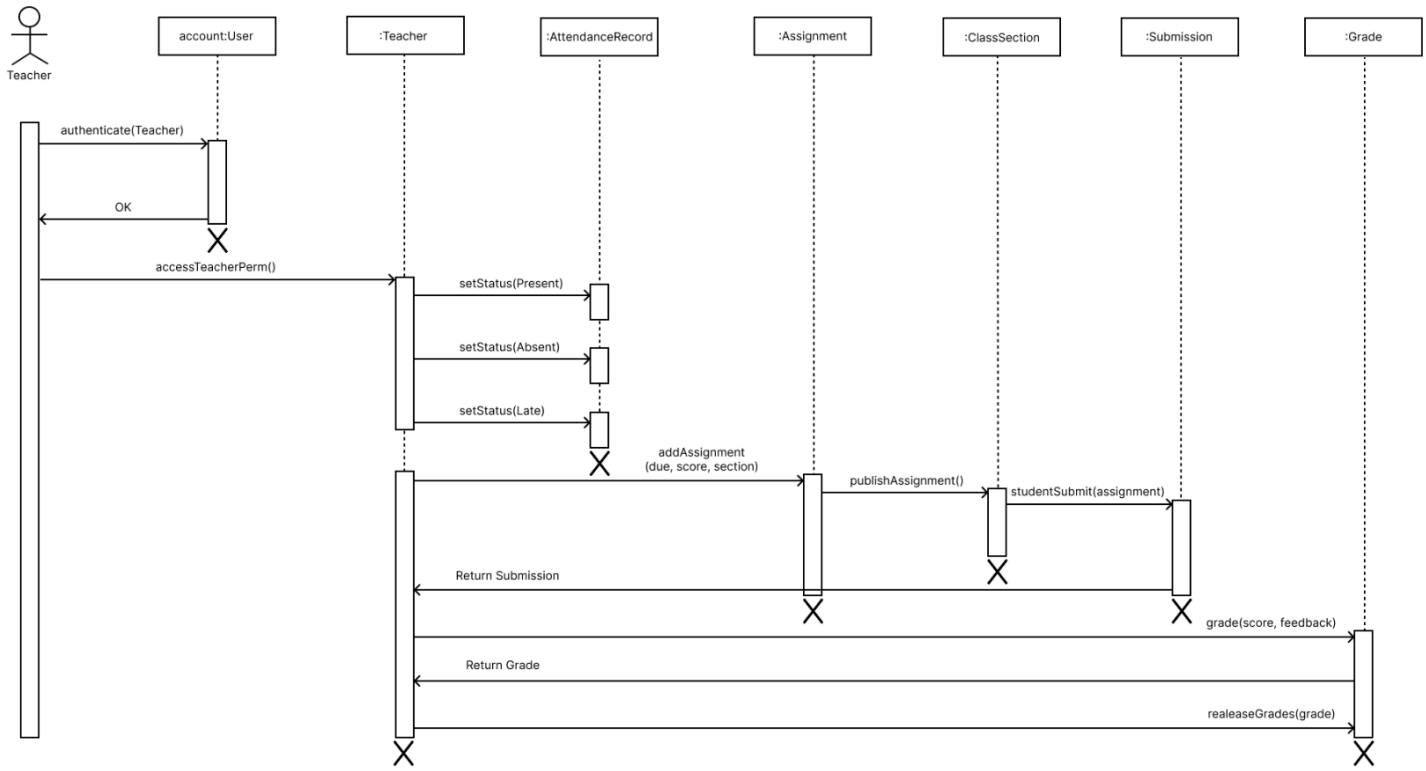


This sequence diagram shows how the School Administrator interacts with the system to manage classes, announcements, and reports.

1. The School Administrator first authenticates through the User account to gain access permissions.
2. After logging in, the admin sends a request to the ClassSection to create a new Class Section, and once it is created, the section is returned.
3. The admin then assigns a teacher to that section using the setTeacher() method.
4. Next, the admin can create and publish announcements to notify teachers.

5. Finally, the admin can view academic reports by calling getReport() from the School object, which returns the requested report.

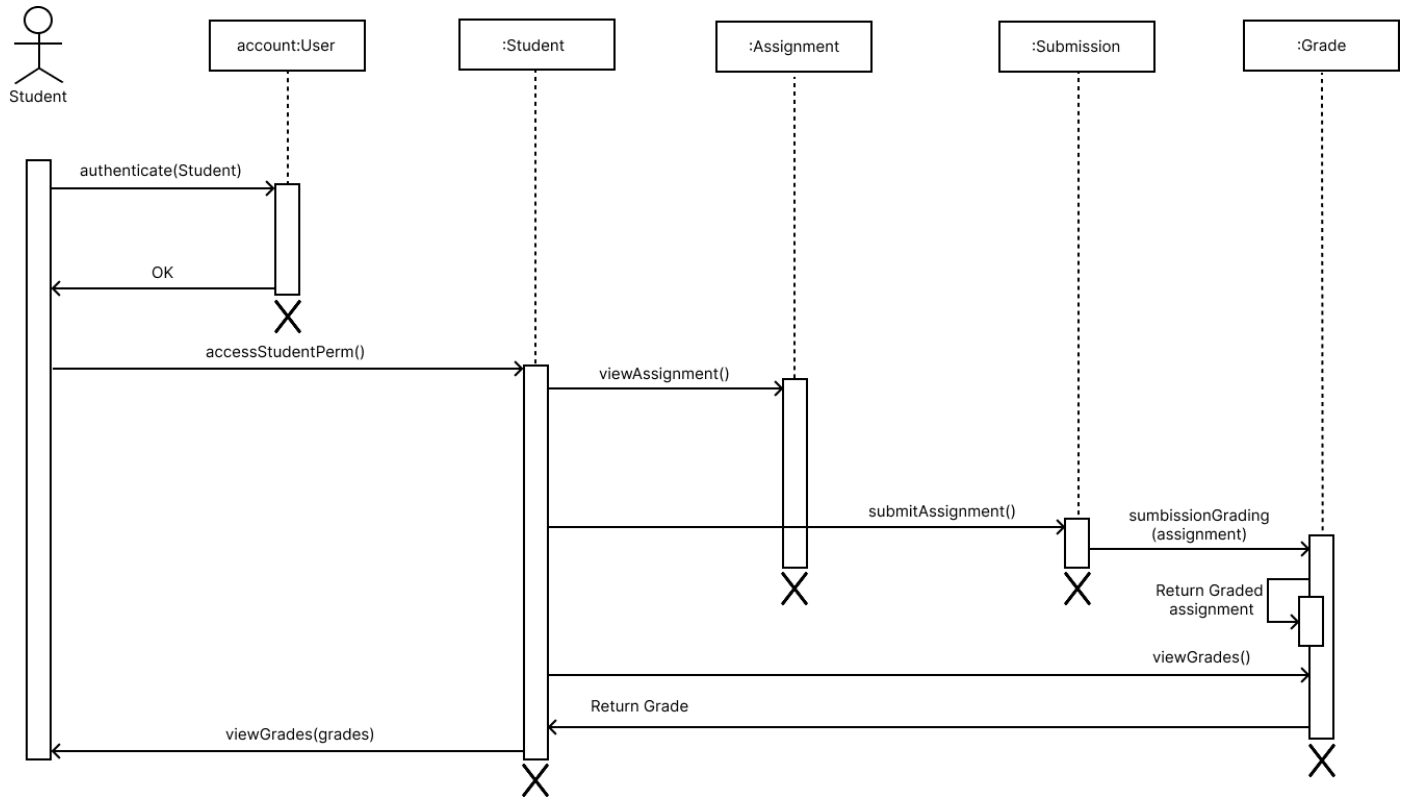
Teacher



This sequence diagram shows how the Teacher interacts with the EduGate system to manage attendance, assignments, and grading.

1. The Teacher first authenticates through the User account to gain system access and permissions.
2. Once logged in, the teacher uses the AttendanceRecord class to set the status of students (Present, Absent, or Late) for a given session.
3. The teacher then creates a new Assignment, associates it with a ClassSection, and publishes it to students.
4. After students submit their work, the teacher retrieves their Submission, assigns a grade with feedback using the grade(score, feedback) method, and finally releases the grades to make them visible to students and parents.

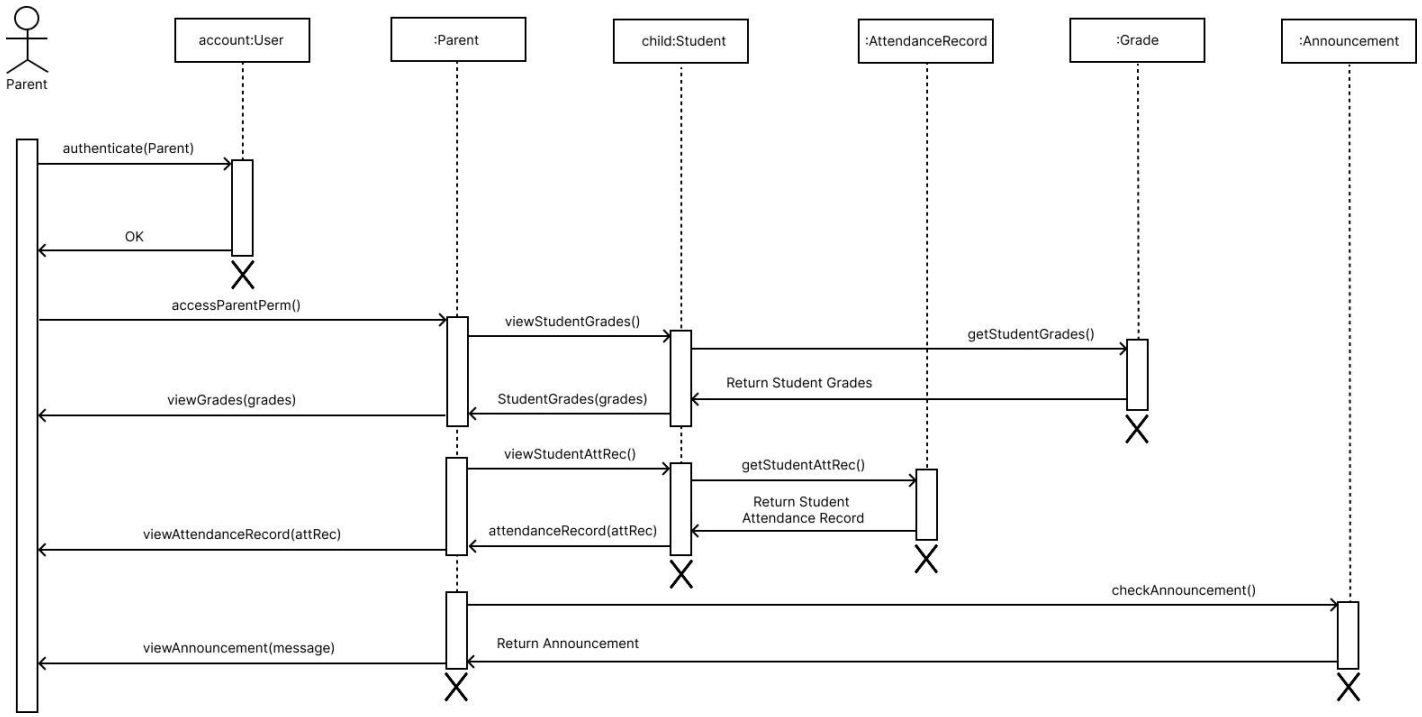
Student



This sequence diagram illustrates how the Student interacts with the EduGate system to manage assignments and grades.

1. The Student starts by authenticating through the User account to gain access to the system.
2. Once logged in, the student uses the viewAssignment() function to view the list of available assignments provided by teachers.
3. The student then submits the completed assignment through the submitAssignment() operation, which creates a new Submission object linked to the specific assignment.
4. After the teacher grades the submission, the student retrieves the result by calling viewGrades(), which accesses the Grade object and returns the graded assignment and feedback.

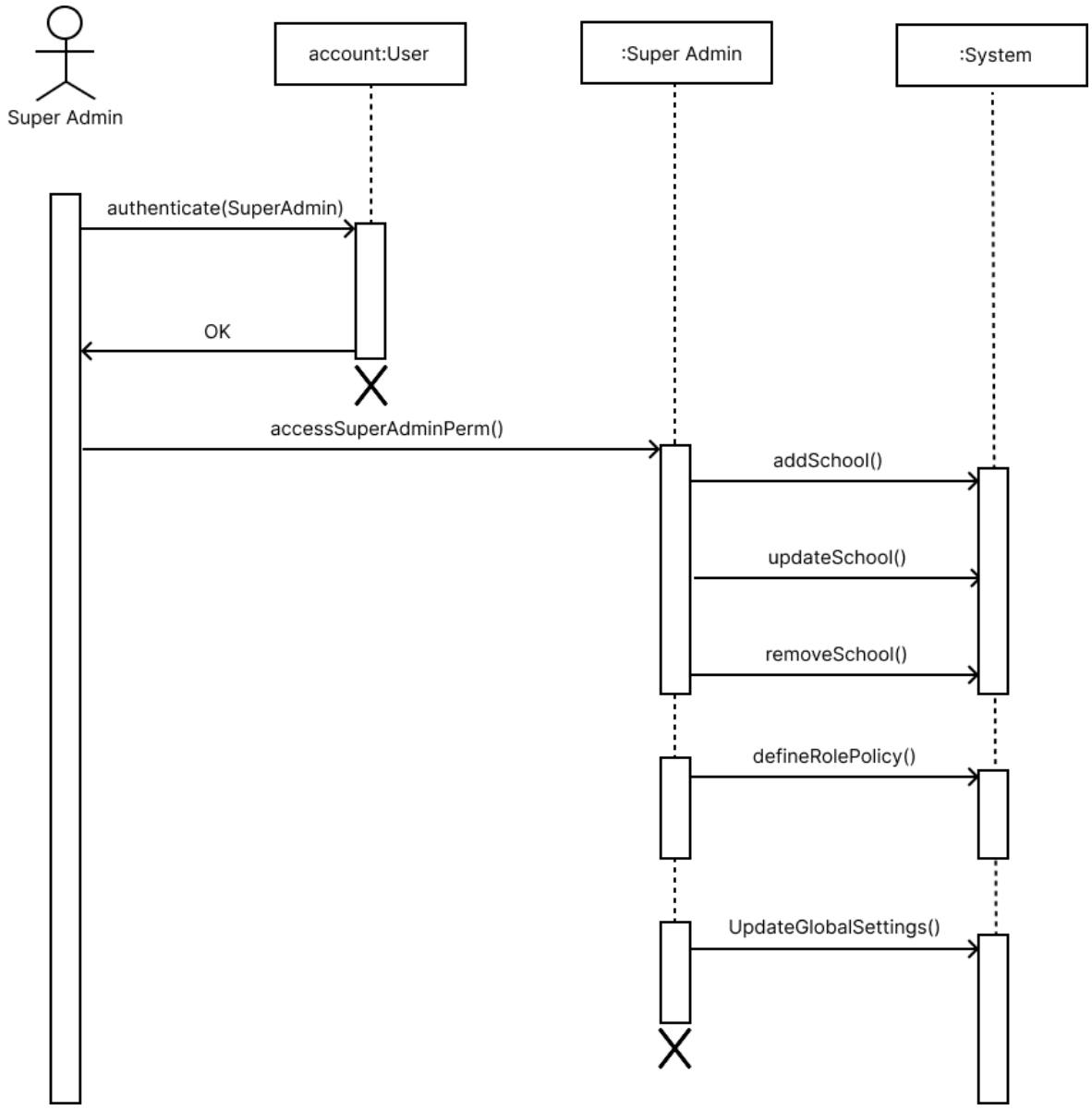
Parent



This sequence diagram demonstrates how the Parent interacts with the EduGate system to monitor their child's progress and stay informed about school updates.

1. The Parent first authenticates through the User account to verify identity and gain access.
2. After logging in, the system checks for parental permissions using accessParentPerm().
3. The parent requests to view the child's grades, triggering a call to the Student object, which retrieves data from the Grade class and returns the results.
4. The parent can also view the child's attendance record, which is fetched from the AttendanceRecord class linked to the student.
5. Lastly, the parent uses viewAnnouncement() to check school or teacher announcements, ensuring they stay updated on important events or messages.

Super Admin

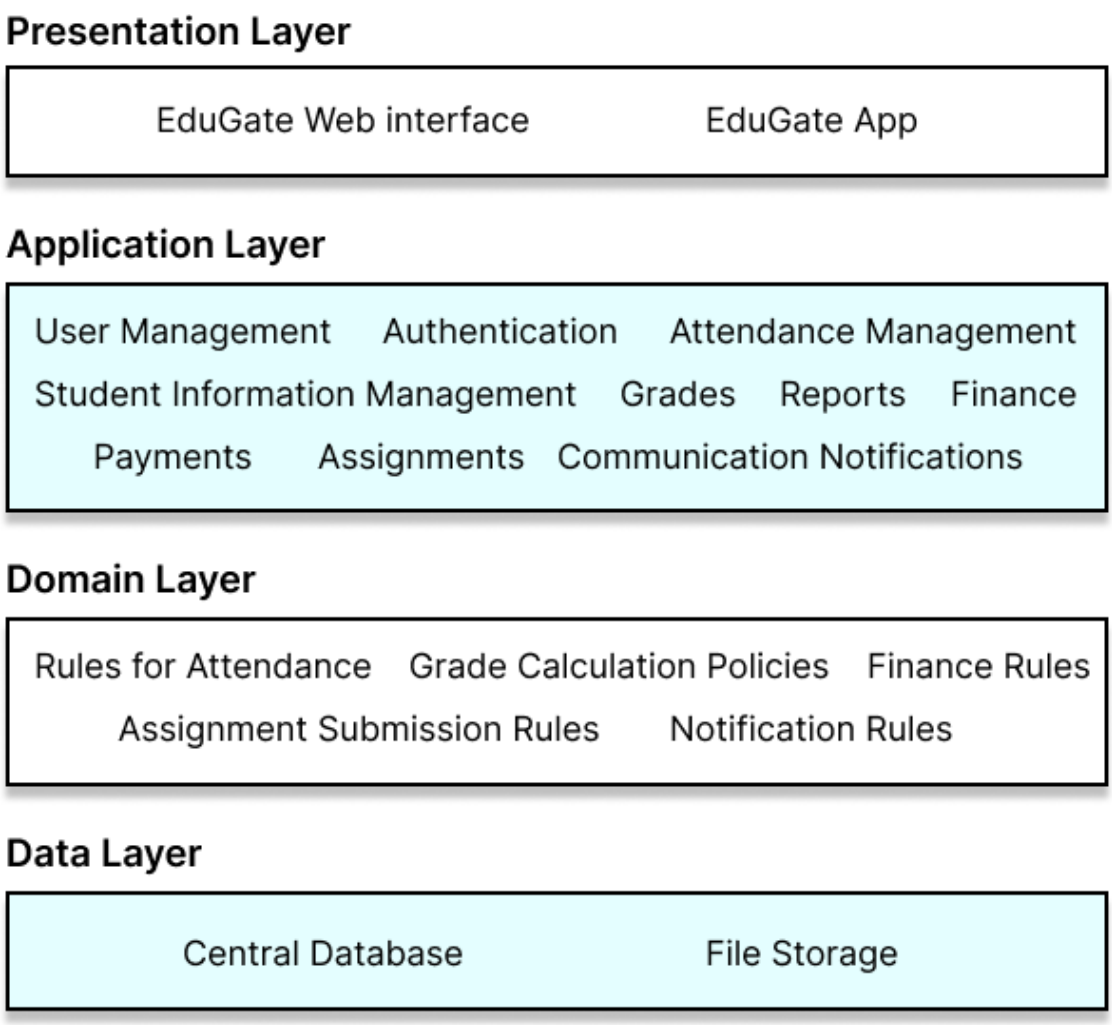


This sequence diagram shows how the Super Admin interacts with the EduGate system to handle global system management tasks.

1. The Super Admin first authenticates through the User account to gain access privileges.
2. After successful login, the system checks for super admin permissions using accessSuperAdminPerm().
3. The Super Admin can then perform high-level operations such as:

- addSchool(), updateSchool(), and removeSchool() managing the schools registered in the EduGate platform.
- defineRolePolicy() creating or updating system-wide roles and permissions.
- updateGlobalSettings() adjusting platform configurations like academic terms or supported languages.

Software Architecture (Layered Architecture)



Layers:

Presentation Layer

This is basically the user interface, everything the user sees and interacts with.

It includes the EduGate web, EduGate app.

What it does: Shows the data in a simple and clear way and takes inputs from the user, like logging attendance, entering grades, or uploading assignments.

It doesn't do any complex processing; it just sends requests to the next layer, the Application Layer.

Application Layer

You can think of this as the system's brain that manages everything between the user and the rest of the system.

Some main tasks:

- Managing users and permissions
- Managing student info
- Handling attendance
- Managing grades and reports
- Managing assignments
- Handling payments and finance
- Sending notifications and messages

What it does: Takes requests from the presentation layer, uses the rules from the domain layer, and updates the data in the database.

Domain Layer

This layer contains the rules and policies of the school system.

Examples:

- How grades are calculated (GPA, average, percentage)

- Attendance rules (when a student is late or absent)
- Assignment rules (deadlines, late penalties, cheating)
- Payment rules and receipts
- Notifications (who gets alerts and when)

Why it's separate: If a school rule changes, we can update it here without touching the other layers.

Data Layer

This is where all the system's data is stored.

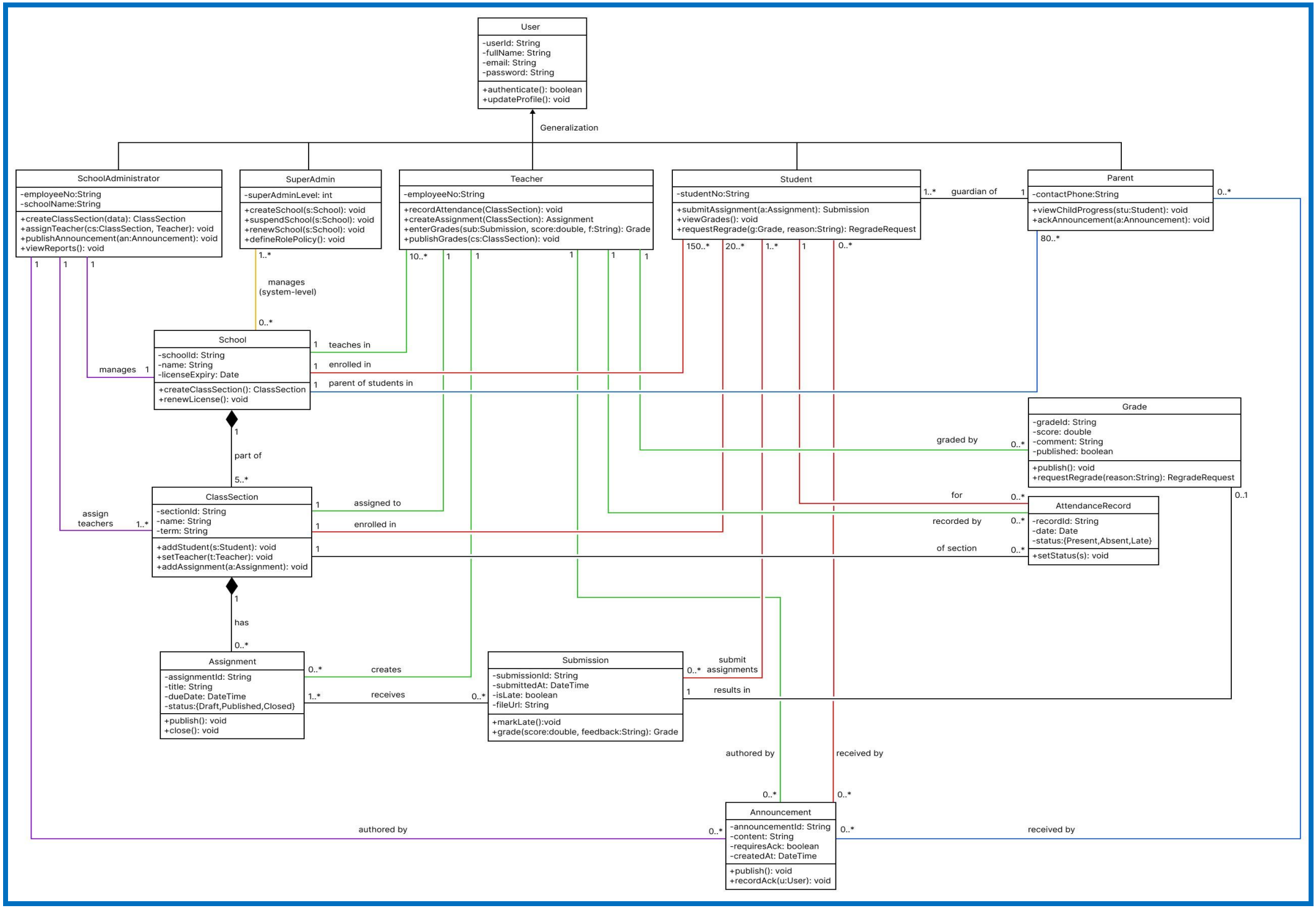
Includes:

- Main database (student info, grades, attendance, payments)
- Cloud storage for files (assignments, reports, documents)

What it does: Keeps data safe and organized and provides it when the application or domain layer asks for it.

Why Layered Architecture is the Best Choice

- Each layer has a clear role, so it's easier to understand and manage.
- Changing something in one layer (like design or rules) doesn't break the rest of the system.
- Some layers, like domain and data, can be reused in other parts of the system.
- It makes adding new features easier.
- Testing and fixing problems is simpler because you can check each layer separately.


User
-userId: String
-fullName: String
-email: String
-password: String
+authenticate(): boolean
+updateProfile(): void
Generalization
SchoolAdministrator
-employeeNo:String
-schoolName:String
+createClassSection(data): ClassSection
+assignTeacher(cs:ClassSection, Teacher): void
+publishAnnouncement(an:Announcement): void
+viewReports(): void
SuperAdmin
-superAdminLevel: int
+createSchool(s:School): void
+suspendSchool(s:School): void
+renewSchool(s:School): void
+defineRolePolicy(): void
Teacher
-employeeNo:String
+recordAttendance(ClassSection): void
+createAssignment(ClassSection): Assignment
+enterGrades(sub:Submission, score:double, f:String): Grade
+publishGrades(cs:ClassSection): void
Student
-studentNo:String
+submitAssignment(a:Assignment): Submission
+viewGrades(): void
+requestRegrade(g:Grade, reason:String): RegradeRequest
Parent
-contactPhone:String
+viewChildProgress(stu:Student): void
+ackAnnouncement(a:Announcement): void
1..*
guardian of
1
0..*
1
1
1
1..*
manages
(system-level)
0..*
10..*
1
1
1
1
1
150..*
20..*
1..*
1
0..*
80..*
School
-schoolId: String
-name: String
-licenseExpiry: Date
+createClassSection(): ClassSection
+renewLicense(): void
1
teaches in
1
enrolled in
1
parent of students in
manages
1
1
part of
5..*
Grade
-gradeId: String
-score: double
-comment: String
-published: boolean
+publish(): void
+requestRegrade(reason:String): RegradeRequest
graded by
0..*
for
0..*
0..1
ClassSection
-sectionId: String
-name: String
-term: String
+addStudent(s:Student): void
+setTeacher(t:Teacher): void
+addAssignment(a:Assignment): void
assign
teachers
1..*
1
assigned to
1
enrolled in
1
AttendanceRecord
-recordId: String
-date: Date
-status:{Present,Absent,Late}
+setStatus(s): void
recorded by
0..*
of section
0..*
1
has
0..*
Assignment
-assignmentId: String
-title: String
-dueDate: DateTime
-status:{Draft,Published,Closed}
+publish(): void
+close(): void
0..*
creates
1..*
receives
0..*
Submission
-submissionId: String
-submittedAt: DateTime
-isLate: boolean
-fileUrl: String
+markLate():void
+grade(score:double, feedback:String): Grade
0..*
submit
assignments
1
results in
authored by
received by
0..*
0..*
Announcement
-announcementId: String
-content: String
-requiresAck: boolean
-createdAt: DateTime
+publish(): void
+recordAck(u:User): void
authored by
0..*
0..*
received by

Class Diagram

School Administrator	
Super Admin	
Teacher	
Student	
Parent	
Other	

User

Represents any system account. All other user types inherit from this class.
Key attributes: userId, fullName, email, passwordHash.
Methods:

- **authenticate(): boolean**. Verifies credentials and returns true if the login is valid.
- **updateProfile(): void**. Lets the user change personal information such as name or email.

SuperAdmin (inherits from User)

System administrator who controls schools at the platform level without accessing private school data.
Key attribute: superAdminLevel.
Methods:

- **createSchool(s: School): void**. Registers a new school in the system.
- **suspendSchool(s: School): void**. Temporarily disables a school record.
- **renewSchool(s: School): void**. Extends or reactivates a school's license.
- **defineRolePolicy(): void**. Creates or updates global roles and permissions.

SchoolAdministrator (inherits from User)

Administrator responsible for one specific school.
Key attributes: employeeNo, schoolName.
Methods:

- **createClassSection(data): ClassSection**. Opens a new class or section for the school.
- **assignTeacher(cs: ClassSection, t: Teacher): void**. Assigns a teacher to a section.

- **publishAnnouncement(a: Announcement): void**. Sends a school announcement to recipients.
- **viewReports(): void**. Retrieves school dashboards or reports for review.

Teacher (inherits from User)

Academic staff member who teaches classes, creates assignments, and grades work.

Key attribute: employeeNo.

Methods:

- **recordAttendance(cs: ClassSection): void**. Records Present, Absent, or Late for students in a section.
- **createAssignment(cs: ClassSection): Assignment**. Creates an assignment and links it to a class.
- **enterGrades(sub: Submission, score: double, comment: String): Grade**. Grades a submission and returns the created or updated Grade.
- **publishGrades(cs: ClassSection): void**. Makes grades visible to students and parents.

Student (inherits from User)

Learner enrolled in one or more class sections.

Key attribute: studentNo.

Methods:

- **submitAssignment(a: Assignment): Submission**. Uploads work and creates a submission record.
- **viewGrades(): void**. Opens the list of published grades and feedback.
- **requestRegrade(g: Grade, reason: String): RegradeRequest**. Files an appeal for a grade.

Parent (inherits from User)

Guardian who can view the child's progress and acknowledge messages.

Key attribute: contactPhone.

Methods:

- **viewChildProgress(stu: Student): void**. Shows the child's attendance and grades.
- **acknowledgeAnnouncement(a: Announcement): void**. Confirms that the parent has read an announcement when acknowledgment is required.

School

Top entity for a single institution. Owns sections and users who belong to it.
Key attributes: schoolId, name, licenseExpiry.
Methods:

- **createClassSection(): ClassSection**. Creates a new ClassSection that belongs to the school.
- **renewLicense(): void**. Updates the school's license validity.

ClassSection

A specific class or section within a school. Owns assignments and enrolls students.
Key attributes: sectionId, name, term.
Methods:

- **addStudent(s: Student): void**. Enrolls a student in the section.
- **setTeacher(t: Teacher): void**. Sets the lead teacher for the section.
- **addAssignment(a: Assignment): void**. Adds an assignment to the section's workload.

Assignment

Work created by a teacher for a section with a due date and publishing state.
Key attributes: assignmentId, title, dueDate, status {Draft, Published, Closed}.
Methods:

- **publish(): void**. Makes the assignment visible to students and triggers notifications.
- **close(): void**. Closes the assignment so no new submissions are accepted.

Submission

A student's delivered work for a specific assignment.

Key attributes: submissionId, submittedAt, isLate, fileUrl.

Methods:

- **markLate(): void**. Flags the submission as late according to due date rules.
- **grade(score: double, comment: String): Grade**. Stores teacher feedback and score, returns the Grade.

Grade

The evaluation for a single submission.

Key attributes: gradeId, score, comment, published.

Methods:

- **publish(): void**. Makes the grade visible to the student and parent.
- **requestRegrade(reason: String): RegradeRequest**. Opens an appeal linked to this grade.

AttendanceRecord

Daily or session attendance for a student within a section.

Key attributes: recordId, date, status {Present, Absent, Late}.

Method:

- **setStatus(s): void**. Sets or updates the attendance value for that record.

Announcement

Message authored by a principal or teacher and delivered to recipients.

Key attributes: announcementId, content, requiresAck, createdAt.

Methods:

- **publish(): void**. Sends the announcement to target users.
- **recordAcknowledgment(u: User): void**. Stores that a specific user has acknowledged the message

Implementation

In the image below you can see the four core methods implementations (**CreateClassSection**, **recordAttendance**, **submitAssignment**, and **PublishAnnouncement**).

EduGate.java × EduGateTest.java ×

Source History

```java
package edugate;

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDate;
import java.time.LocalDateTime;

public class EduGate {

    public static void createClassSection(String sectionId, String name,String term, String teacherId) {

        try (PrintWriter out = new PrintWriter(new FileWriter("class_sections.txt", true))) {
            out.println(sectionId + "," + name + "," + term + "," + teacherId);
        } catch (IOException e) {
            System.out.println("Error saving class section: " + e.getMessage());
        }

        System.out.println("Class section created successfully.");
    }

    public static void recordAttendance(String studentId, String sectionId, LocalDate date, String status) {

        try (PrintWriter out = new PrintWriter(new FileWriter("attendance_records.txt", true))) {
            out.println(studentId + "," + sectionId + "," + date + "," + status);
        } catch (IOException e) {
            System.out.println("Error saving attendance: " + e.getMessage());
        }

        System.out.println("Attendance recorded.");
    }

    public static void submitAssignment(String assignmentId, String studentId, LocalDateTime dueDate, String filePath) {

        LocalDateTime submittedAt = LocalDateTime.now();
        boolean late = submittedAt.isAfter(dueDate);

        try (PrintWriter out = new PrintWriter(new FileWriter("submissions.txt", true))) {
            out.println(studentId + "," + assignmentId + "," + submittedAt + "," + late + "," + filePath);
        } catch (IOException e) {
            System.out.println("Error saving submission: " + e.getMessage());
        }

        if (late) {
            System.out.println("Submission received but marked as LATE.");
        } else {
            System.out.println("Submission received on time.");
        }
    }

    public static void publishAnnouncement(String announcementId, String authorId, String content, boolean requiresAck) {

        LocalDateTime createdAt = LocalDateTime.now();

        try (PrintWriter out = new PrintWriter(new FileWriter("announcements.txt", true))) {
            out.println(announcementId + "," + authorId + "," + createdAt + "," + requiresAck + "," + content);
        } catch (IOException e) {
            System.out.println("Error saving announcement: " + e.getMessage());
        }

        System.out.println("Announcement published.");
    }
```

The methods running on example inputs:

```
    public static void main(String[] args) {
    EduGate.createClassSection("SEC1 ", "1A ", "Term1 ", "T10");

    EduGate.recordAttendance("S1 ", "SEC1 ", LocalDate.now(), " Present");

    LocalDateTime due = LocalDateTime.now().plusDays(1);
    EduGate.submitAssignment("A1 ", "S1 ", due, " onTime.pdf");

    LocalDateTime dueLate = LocalDateTime.now().minusDays(1);
    EduGate.submitAssignment("A2", "S1", dueLate, "lateFile.pdf");

    EduGate.publishAnnouncement("AN1 ", "ADMIN1 ", " Tomorrow is a holiday.", false);
    }
}
```

The outputs generated by the code:

EduGate.java × | EduGateTest.java ×

Source | History

```java
package edugate;

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDate;
import java.time.LocalDateTime;

public class EduGate {

    public static void createClassSection(String sectionId, String name,String term, String teacherId) {

        try (PrintWriter out = new PrintWriter(new FileWriter("class_sections.txt", true))) {
            out.println(sectionId + "," + name + "," + term + "," + teacherId);
        } catch (IOException e) {
            System.out.println("Error saving class section: " + e.getMessage());
        }

        System.out.println("Class section created successfully.");
    }

    public static void recordAttendance(String studentId, String sectionId, LocalDate date, String status) {

        try (PrintWriter out = new PrintWriter(new FileWriter("attendance_records.txt", true))) {
            out.println(studentId + "," + sectionId + "," + date + "," + status);
        } catch (IOException e) {
            System.out.println("Error saving attendance: " + e.getMessage());
        }

        System.out.println("Attendance recorded.");
    }

    public static void submitAssignment(String assignmentId, String studentId, LocalDateTime dueDate, String filePath) {

        LocalDateTime submittedAt = LocalDateTime.now();
        boolean late = submittedAt.isAfter(dueDate);

        try (PrintWriter out = new PrintWriter(new FileWriter("submissions.txt", true))) {
            out.println(studentId + "," + assignmentId + "," + submittedAt + "," + late + "," + filePath);
        } catch (IOException e) {
            System.out.println("Error saving submission: " + e.getMessage());
        }

        if (late) {
            System.out.println("Submission received but marked as LATE.");
        } else {
            System.out.println("Submission received on time.");
        }
    }

    public static void publishAnnouncement(String announcementId, String authorId, String content, boolean requiresAck) {

        LocalDateTime createdAt = LocalDateTime.now();

```

Output - EduGate (run)

```
run:
Class section created successfully.
Attendance recorded.
Submission received on time.
Submission received but marked as LATE.
Announcement published.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Testing

EduGate.java × EduGateTest.java ×

Source History

```java
package edugate;

import java.io.IOException;
import java.nio.file.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.*;

public class EduGateTest {

    @Before
    public void cleanUpFiles() throws IOException {
        Files.deleteIfExists(Paths.get("class_sections.txt"));
        Files.deleteIfExists(Paths.get("attendance_records.txt"));
        Files.deleteIfExists(Paths.get("submissions.txt"));
        Files.deleteIfExists(Paths.get("announcements.txt"));
    }

    @Test
    public void testCreateClassSection_writesToFile() throws Exception {
        EduGate.createClassSection("SEC1", "1A", "Term1", "T10");

        Path path = Paths.get("class_sections.txt");
        assertTrue(Files.exists(path));

        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("SEC1"));
        assertTrue(last.contains("1A"));
        assertTrue(last.contains("Term1"));
        assertTrue(last.contains("T10"));
    }

    @Test
    public void testRecordAttendance_present() throws Exception {
        LocalDate date = LocalDate.of(2025, 11, 27);

        EduGate.recordAttendance("S1", "SEC1", date, "Present");

        Path path = Paths.get("attendance_records.txt");
        assertTrue(Files.exists(path));

        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("S1"));
        assertTrue(last.contains("SEC1"));
        assertTrue(last.contains(date.toString()));
        assertTrue(last.contains("Present"));
    }
```

```java
    @Test
    public void testRecordAttendance_absent() throws Exception {
        LocalDate date = LocalDate.of(2025, 11, 28);

        EduGate.recordAttendance("S2", "SEC1", date, "Absent");

        Path path = Paths.get("attendance_records.txt");
        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("S2"));
        assertTrue(last.contains("SEC1"));
        assertTrue(last.contains(date.toString()));
        assertTrue(last.contains("Absent"));
    }

    @Test
    public void testSubmitAssignment_onTime() throws Exception {
        LocalDateTime dueDate = LocalDateTime.now().plusDays(1);

        EduGate.submitAssignment("A1", "S1", dueDate, "file.pdf");

        Path path = Paths.get("submissions.txt");
        assertTrue(Files.exists(path));

        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("S1"));
        assertTrue(last.contains("A1"));
        assertTrue(last.contains("file.pdf"));
        assertTrue(last.contains("false")); // Not late
    }

    @Test
    public void testSubmitAssignment_late() throws Exception {
        LocalDateTime dueDate = LocalDateTime.now().minusDays(1);

        EduGate.submitAssignment("A2", "S2", dueDate, "late.pdf");

        Path path = Paths.get("submissions.txt");
        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("S2"));
        assertTrue(last.contains("A2"));
        assertTrue(last.contains("late.pdf"));
        assertTrue(last.contains("true")); // Late
    }
```

```java
    @Test
    public void testPublishAnnouncement_basic() throws Exception {
        EduGate.publishAnnouncement("AN1", "ADMIN1", "Tomorrow is a holiday.", false);

        Path path = Paths.get("announcements.txt");
        assertTrue(Files.exists(path));

        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("AN1"));
        assertTrue(last.contains("ADMIN1"));
        assertTrue(last.contains("false"));
        assertTrue(last.contains("Tomorrow is a holiday."));
    }

    @Test
    public void testPublishAnnouncement_requiresAck() throws Exception {
        EduGate.publishAnnouncement("AN2", "ADMIN1", "Please acknowledge this message.", true);

        Path path = Paths.get("announcements.txt");
        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("AN2"));
        assertTrue(last.contains("ADMIN1"));
        assertTrue(last.contains("true"));
        assertTrue(last.contains("Please acknowledge this message."));
    }
}
```

Testing result:

EduGate.java × EduGateTest.java ×

Source History

```java
package edugate;

import java.io.IOException;
import java.nio.file.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.*;

public class EduGateTest {

    @Before
    public void cleanUpFiles() throws IOException {
        Files.deleteIfExists(Paths.get("class_sections.txt"));
        Files.deleteIfExists(Paths.get("attendance_records.txt"));
        Files.deleteIfExists(Paths.get("submissions.txt"));
        Files.deleteIfExists(Paths.get("announcements.txt"));
    }

    @Test
    public void testCreateClassSection_writesToFile() throws Exception {
        EduGate.createClassSection("SEC1", "1A", "Term1", "T10");

        Path path = Paths.get("class_sections.txt");
        assertTrue(Files.exists(path));

        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("SEC1"));
        assertTrue(last.contains("1A"));
        assertTrue(last.contains("Term1"));
        assertTrue(last.contains("T10"));
    }

    @Test
    public void testRecordAttendance_present() throws Exception {
        LocalDate date = LocalDate.of(2025, 11, 27);

        EduGate.recordAttendance("S1", "SEC1", date, "Present");

        Path path = Paths.get("attendance_records.txt");
        assertTrue(Files.exists(path));

        List<String> lines = Files.readAllLines(path);
        assertFalse(lines.isEmpty());

        String last = lines.get(lines.size() - 1);
        assertTrue(last.contains("S1"));
```

Test Results

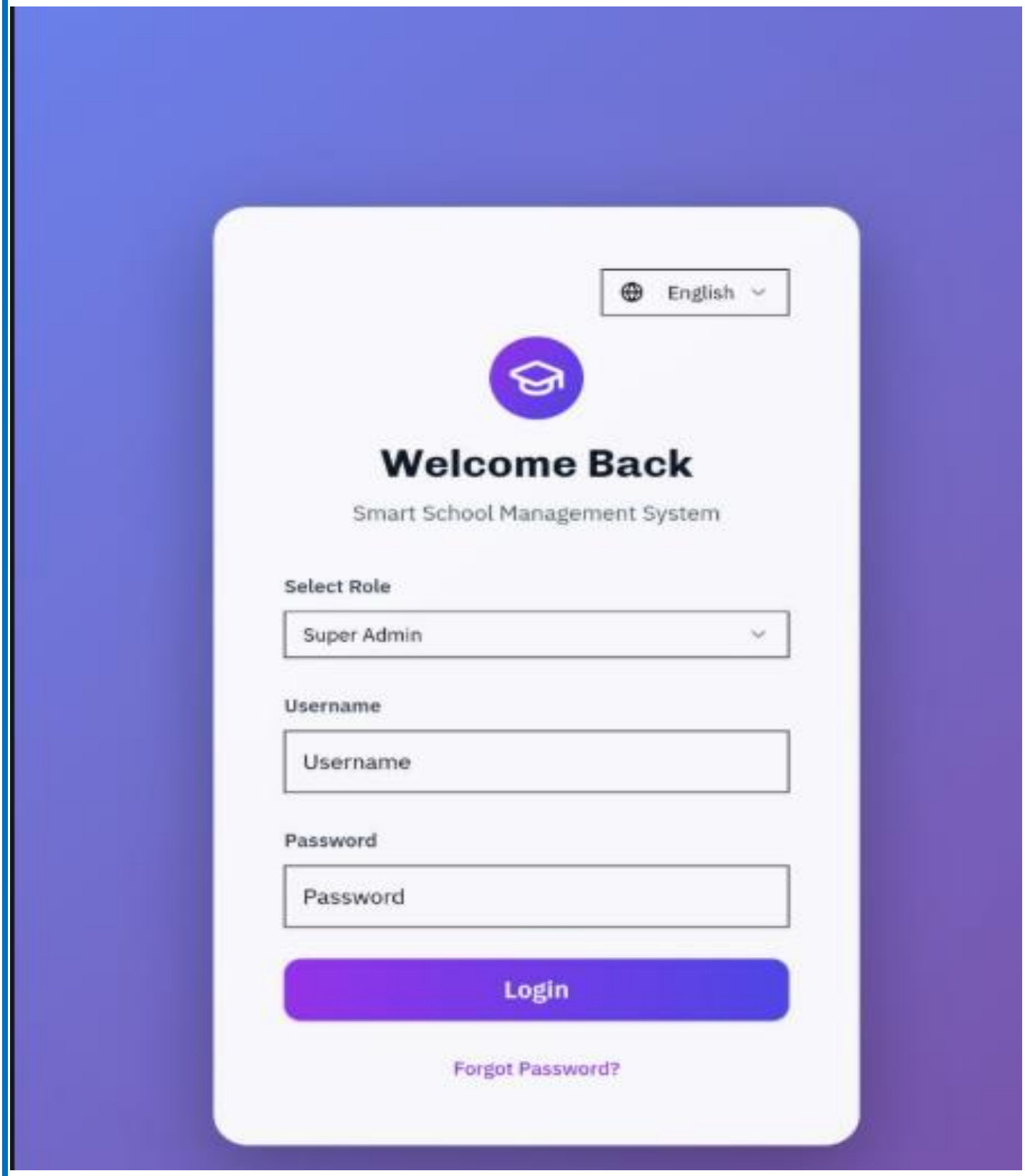
edugate.EduGateTest ×

Tests passed: 100.00 %

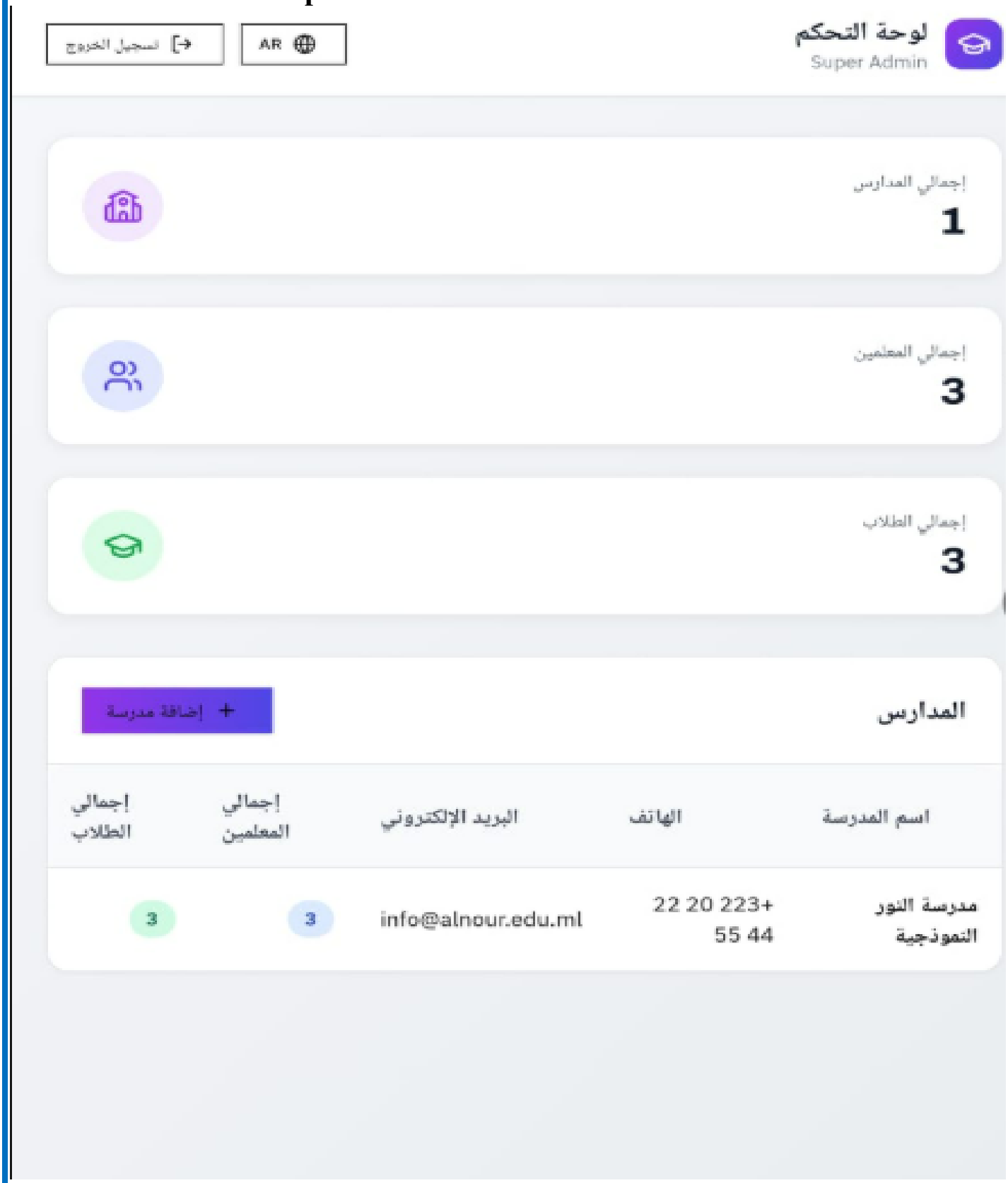
All 7 tests passed. (0.107 s)

```
Class section created successfully.
Submission received on time.
Attendance recorded.
Announcement published.
Attendance recorded.
Submission received but marked as LATE.
Announcement published.
```

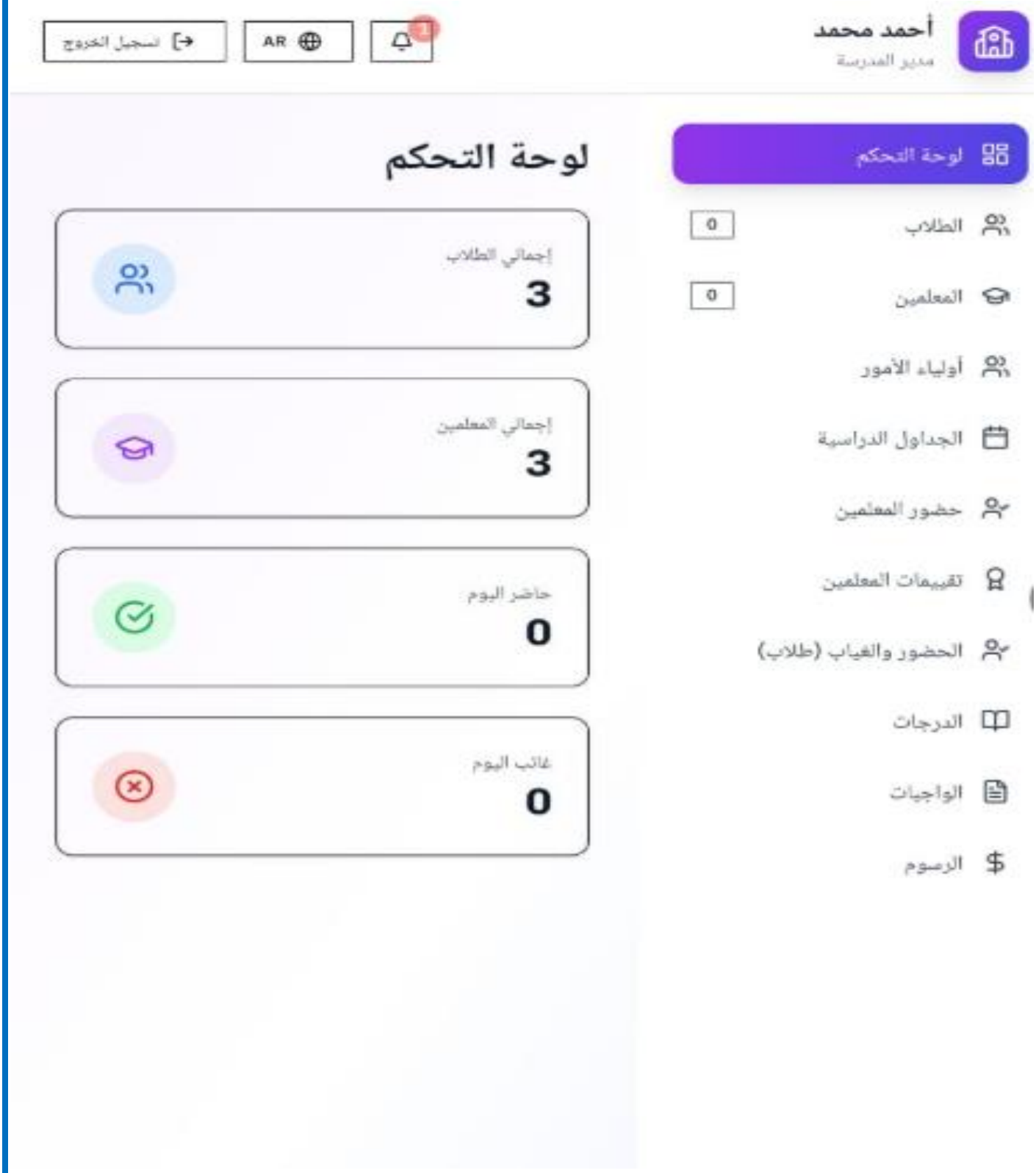
User Interface Loging:



User Interface Super Admin:



User Interface School Manager:



Team Synergy & Work Distribution

Project Philosophy: One Team, One Vision The development of "EduGate" was defined by a spirit of unified collaboration. While individual members spearheaded specific execution tasks, the "collective mind" was the driving force behind every major decision. We operated with fluid communication, ensuring that every part of the system reflected our shared understanding.

- **Ideation & Vision (The Spark):** The project's concept was born from intense, participatory brainstorming sessions involving all members. After exchanging various proposals, we unanimously selected the idea presented by **Abdoul Malick Cisse**. We chose this concept not just for its technical potential, but for its profound humanitarian value and realistic solutions for educational challenges in developing regions like Mali. The team immediately adopted this vision as our own.

- **Requirements Gathering & Development Approach:** This phase was entirely a group effort. The entire team participated in gathering requirements, defining the system's scope, and selecting the **Agile methodology**. Every requirement listed was the result of mutual debate and idea exchange to ensure we covered all functional and non-functional needs.

- **System Modeling & Architecture (The Architects):** Translating our abstract ideas into concrete technical blueprints was led by **Munthir Alfarsi**, **Hashim Bajaber**, and **Abdulaziz Melebari**. They were responsible for designing the system's visual logic, including the Use Case, Sequence, and Class Diagrams. However, these diagrams were subject to collective review to ensure they perfectly aligned with the intended business logic.

- **Implementation, UI & Testing (The Builders):** The technical realization was driven by **Abdoul Malick Cisse** and **Rami Alyazedy**. They handled the heavy lifting of the UI design, Java implementation, and Unit Testing. They ensured the code strictly adhered to the Layered Architecture, with constant feedback from the rest of the team to verify that the logic matched our design models.

Challenges & Lessons Learned

Challenges: As students, our biggest challenge was bridging the gap between theory and practice:

- **Mapping Diagrams to Code:** We faced difficulty in strictly following the Class Diagrams during implementation. Translating the visual relationships into actual Java code required us to go back and forth between the design and the code multiple times.

- **Data Persistence (File Handling):** Since we haven't integrated a full database yet, managing data using text files (.txt) was tricky. We had to be careful about how data was written and read to avoid errors during our tests.

- **Coordination:** Coordinating our schedules and merging our different parts of work was a challenge, but it taught us the importance of communication in software engineering.

Lessons Learned:

- **The Importance of Design:** We learned that spending time on a good design (Layered Architecture) makes coding easier, even for a simple project.

- **Agile Mindset:** We understood how Agile allows us to fix mistakes early. We didn't have to wait until the end to realize something was wrong.

- **Testing Value:** Using JUnit taught us that automated testing is much faster and more reliable than manual checking.

Conclusion & Future Goals

Conclusion: In this project, we successfully created a working prototype of "EduGate," a multilingual school management system. Although it is a simplified version using file storage and basic logic, it represents a significant milestone in our learning journey. We managed to apply the core concepts of CPIT-251, from requirements gathering to testing, while keeping a humanitarian goal in mind: making education management accessible and organized.

Future Educational Goals: As we continue our studies, we plan to use this project as a base to learn more advanced technologies:

- **Database Integration:** We aim to replace the current text-file storage with a real Relational Database (like MySQL) to learn about SQL connectivity and data integrity.
- **User Interface Improvement:** We want to move beyond simple interfaces and learn how to build a modern, user-friendly Web or GUI interface that improves the user experience.
- **Advanced Architecture:** We hope to deepen our understanding of the Layered Architecture by strictly separating the business logic from the user interface in future iterations.